JAVA Programs

Unit 1 :-Basics of OOPL & Class:	
NO.	**Programs List(conditional Statements, Loops, Arrays, Scanner class, Command Line, String and String Buffer method)**
1.	Write Program to Print Student Background details.(Ex. Name, Hobby..)
2.	WAP to find No is Odd and Even.(Take input from command line)
3.	WAP to print Fibonacci series.
4.	WAP to print Factorial. (Take Input from User using Scanner class)
5.	WAP to print No is Palindrome or not. (Take Input from User using Scanner class)
6.	WAP to Check No is Palindrome or Not.
7.	WAP to Check No is Armstrong or Not.
8.	WAP to Find the minimum and maximum Element in an Array
9.	WAP Count Positive, Negative, and Zero Elements in an Array
10.	WAP to Find a Specific Element in an Array.
11.	WAP to take values from user in 2D Array and perform row wise and column wise sum.
12.	WAP to print Reverse string.
13.	WAP to String is Palindrome or Not.
14.	WAP to get string value from user and Count Vowels and Consonants in a String.
15.	Create a class Car with attributes brand and model, and a method to display car details.(Defining a class, creating objects, working with methods)
16.	Create a class Student with a constructor to initialize name and grade. Add a method to print details.(Constructor)
17.	Create a class Employee with private fields name and salary. Use getters and setters to access and modify the fields.
18.	Create a class Calculator with methods for addition, subtraction, multiplication, and division
19.	Create a class Book with overloaded constructors: one with no arguments, one with the title, and another with title and author.

20.	Create a class Person with a copy constructor that duplicates an existing object.(Copy constructor).
21.	Create a class Utility with a static method convertToUpperCase(String) and a static variable counter to track calls.(Static methods and variables).
22.	Create a class MathOperations with overloaded methods for addition (e.g., add(int, int) and add(double, double,int))
23.	Create a class Outer with a nested class Inner that can access the outer class's private fields. (Nested (inner) classes).
24.	Modify the Student class to add default, parameterized & copy constructors. Create three objects of the Student class to test the above three constructors. Print the content of the all the three objects
25.	Create a method greet() in a class Message and call it using an anonymous object.(Anonymous objects)
26.	A car showroom manages details of different cars, including: - Brand & Model → Identifies the car. - Price → Base price before tax and insurance. - On-Road Price Calculation → Final price including tax & insurance. - Method Overloading → Two ways to calculate on-road price: Without insurance (tax only). With insurance (tax + insurance).
27.	A bank account allows customers to: - Deposit Money → Increase account balance. - Withdraw Money → Deduct money from balance if sufficient. -s Check Account Details → View account holder name, number, and balance. We will implement Encapsulation, Parameterized Constructor, and Methods to perform banking operations.
	Unit 2:- Inheritance & Polymorphism:
28.	Create a base class Animal with a method eat(). Create a derived class Dog with an additional method bark()(single Inheritance).
29.	Create a base class Vehicle, a subclass Car, and a subclass of Car called ElectricCar. Demonstrate inheritance across multiple levels.(Multilevel inheritance)
30.	Create a base class Shape with a method area(). Create subclasses Circle and Rectangle that override the area() method.(Hierarchical Inheritance)
31.	Create a base class Appliance and an interface Electric. Create a subclass Fan that inherits from both.(Hybrid inheritance using interfaces)
32.	Create a base class Person with a constructor that initializes name. Create a derived class Employee that uses super to call the base class constructor.(super keyword)
33.	Create a base class Parent with a method display(). Create a derived class Child that overrides display() and uses super to call the parent method.(use super keyword for method)

JAVA Programs

34.	Create a base class Shape with a method draw(). Create subclasses Circle and Rectangle that override draw(). Demonstrate polymorphism by storing subclass objects in a base class reference.(Runtime polymorphism, method overriding)
35.	Create an abstract class Animal with an abstract method sound(). Create subclasses Dog and Cat that implement sound().(Abstract classes, polymorphism)
36.	Create an interface Animal with a method sound(). Implement the interface in classes Dog and Cat. Use polymorphism to call the sound() method.(Polymorphism using interfaces)
37.	Create a base class Product with an instance variable price. Create a subclass Laptop that has its own price and uses super.price to differentiate between parent and child class variables.(Using `super` with instance variables).
38.	Create a base class Payment and subclasses CreditCardPayment and CashPayment. Create a method processPayment(Payment payment) that accepts the base class but works with subclass objects.(Polymorphism with method parameters)
39.	Create a base class Animal and subclasses Dog and Cat. Demonstrate upcasting (Animal a = new Dog()) and downcasting (Dog d = (Dog)a).(Upcasting and downcasting in inheritance)
40.	Create a base class Device with a final method turnOn(). Demonstrate that the method cannot be overridden in a subclass.(Final methods in inheritance)
41.	Write program for Sports Tournament Management and Sport interface is implemented by Cricket, Football, and Tennis.
42.	A Smartphone has different functionalities, such as: -Calling Feature (Making and receiving calls) -Camera Feature (Taking photos and recording videos) Since a smartphone supports multiple functionalities, we can use multiple interfaces instead of a single inheritance structure.
43.	A Library consists of different types of books: Physical Books → Books that can be borrowed in person. Digital Books → E-books that can be read online. Since both PhysicalBook and DigitalBook share common properties like title, author, and ISBN, we create a Book superclass. However, both need borrowing functionality, so we define a Borrowable interface with methods like borrowBook() and returnBook().
44.	Write program for Person Greeting System (Method Overloading)
45.	A train ticket booking system where different ticket types (General, Tatkal, Premium) have different booking charges and processing times. - Approach: Create a parent class TrainTicket → Defines a common bookTicket() method. Create child classes (GeneralTicket, TatkalTicket, PremiumTicket) → Override bookTicket() for different behaviors.
46.	A Car Rental System calculates the rental price based on: -Car Type → SUV, Sedan, or Hatchback.

	-Base Fare → Each car type has a different rental rate per day. - Additional Charges → SUVs may have extra charges due to fuel efficiency. Since all cars share a common property (calculateRent() method) but with different pricing, Method Overriding is the best approach.
47.	A Vehicle Fuel Consumption System determines fuel efficiency based on: - Car Type → Petrol, Diesel, or Electric. - Fuel Efficiency (Mileage) → Measured in km/l (for Petrol & Diesel) and km per charge (for Electric Cars). - Fuel Cost Calculation → Different formulas for different car types. Since all vehicles share a common method (calculateFuelEfficiency()), but each type has a different way of computing fuel consumption, we use Method Overriding.
48.	An electricity billing system where bills are calculated differently for Residential, Commercial, and Industrial users. - Approach: Create a parent class ElectricityBill → Defines a generic calculateBill() method. Create child classes (ResidentialBill, CommercialBill, IndustrialBill) → Override calculateBill() with different tariff rates.
	Unit 3:-Interfaces, Exception & Collection:
49.	Create an interface Animal with methods sound() and eat(). Implement it in classes Dog and Cat.
50.	Create Interface demo1 and demo 2 use properties and method in demo3 interface (Multiple Interface)
51.	Create an interface Shape with a method draw(). Create another interface ColoredShape that extends Shape and adds a method color(). Implement ColoredShape in a class Circle(Interface Inheritance)
52.	Create an abstract class Shape with an abstract method area(). Create subclasses Circle and Rectangle that implement area().(Abstract class and method implementation)
53.	WAP to Divide two numbers. Use try-catch to handle ArithmeticException if the divisor is zero.(Try ,Catch and finally block)
54.	Perform operations like array access and division. Use multiple catch blocks for ArithmeticException and ArrayIndexOutOfBoundsException.(Handling multiple exceptions)
55.	WAP Handle exceptions in nested try blocks, such as an inner block for array access and an outer block for arithmetic operations.(Nested try-catch.)
56.	Create a custom method validateAge(int age) that throws an IllegalArgumentException if age is less than 18.(Using the `throw` keyword to explicitly throw an exception.)
57.	Create a class BankAccount with a constructor that throws InvalidBalanceException if the initial balance is negative.(Using `throws` with constructors)
58.	Create a Collection of String objects using ArrayList. Add, remove, and iterate over the elements.(Basic `Collection` operations)

59.	Demonstrate methods from the Collections utility class like sort(), reverse()on a list.
60.	Create Queue of 5 Books. Iterate from first to last book and display all the information about each book.
61.	Create student ArrayLIst and Store and retrieve student names from a list.
62.	Create Employee Hashset Store unique employee IDs to prevent duplicates.(Store unique employee IDs to prevent duplicates.)
63.	Group student and Categorize students based on their grades.(Using HashMap)
64.	Create vector of 5 students. Iterate from first to last student and display all the information about each student.
	Unit 4 Multi-Threading, Generic, I/O
65.	Write program using multiple Thread A student studies while listening to music. Each activity is a separate thread.
66.	Write Program and make Chat Application (Send & Receive Messages in Parallel)using Runnable Interface Simulates a chat app where a user can type messages while receiving them.(Runnable, Thread.sleep()).
67.	Write Program for Creating a Folder for Library Sections(File.mkdir()).
68.	Write Program using Runnable Interface Person Walking and Talking (Two Threads in One Class) Simulates a person performing two actions: walking and talking at the same time.
69.	Write Program for Bank Transactions (Thread Safety and Synchronization) Multiple users deposit and withdraw from a shared bank account, ensuring balance consistency.(`synchronized` methods, Thread safety)
70.	Creating a File for a Store Student Record(using File.createNewFile()).
71.	Creating a Folder for Library Sections. (different sections (e.g., Science, Arts).
72.	Write Program to Renames a file containing a student's report card.
73.	Write Program Updating Student file Name.(using File.renameTo())
74.	Write Program for Reading & Writing Binary Data (Images, Audio, etc.)(using FileInputStream, FileOutputStream)
75.	Write Program for Deleting Old Student Records(using File.delete()).
	Unit 5 GUI Programming with Swing:
76.	Create program for Creating a Simple Swing Window

JAVA Programs

77.	Creates a basic form to enter personal details.(Person Profile Form (AWT Form)(Frame, Label, TextField, Button)
78.	Create program for Student Registration Form and student fills in name, age, and course details.(TextField, Choice, Button)
79.	Create Program for Music Playlist Creator and Users add songs to a playlist using text fields and buttons.(TextField, Button, Choice, Label, Button)
80.	Create program for making Online Quiz System (Swing)and Displays questions and validates user-selected answers.(using JLabel, JRadioButton, JButton)
81.	Create program for Employee Payroll System and Employees enter salary details and tax is calculated.(using JTextField, JButton, JLabel)
82.	Write program for Employee Salary Calculation on Button Click and Calculates and displays salary details on click.(using JButton, ActionListener).
83.	Write program for Gaming Application (Mouse Click Detection) Detects mouse clicks on the game area.(MouseListener)
84.	Write program for Virtual Keyboard (Key Press Handling) using keyListner Displays the typed character when a key is pressed.
85.	Write Program for Factory Machine Control System (Mouse Events for Machine Controls)and Controls a machine when the mouse is clicked on different buttons.(using MouseListener)